

Product Requirements Document (PRD)

Project: Health Risk Dashboard

1. Problem Statement & Objective

Problem: Healthcare datasets contain multiple patient and lifestyle attributes, making it difficult to quickly identify health-risk patterns and understand which factors contribute most to risk.

Objective: Build a local, interactive Health Risk Dashboard that analyzes patient data, visualizes risk factors, predicts a selected health-risk category using machine learning, and provides simple explanations for the prediction.

2. Target Users

- Primary: TY B.Sc. Data Science students and project evaluators
- Secondary: Healthcare analysts, researchers, and students
- End user: A user who wants to explore anonymized healthcare data and understand risk patterns

«The system is for educational/analytical purposes and not for medical diagnosis.»

3. Key Features / Functional Requirements

1. Dashboard Overview

- Total patients
- High/medium/low-risk counts
- Average age, BMI, blood pressure, glucose, etc.

2. Interactive Filtering

- Age group
- Gender
- Risk category
- Lifestyle factors

3. Health Risk Visualization

- Risk distribution
- BMI vs. risk
- Blood pressure vs. risk
- Glucose/cholesterol vs. risk
- Lifestyle vs. risk

4. Risk Prediction

- User enters patient attributes.
- ML model predicts the risk category.

5. Explainable AI

- Display the most influential factors behind a prediction.
- Use SHAP-based feature explanations.

6. Data Upload

- Allow CSV dataset upload.
- Validate required columns before analysis.

7. Report/Insight Section

- Automatically summarize major trends.
- Optional local LLM can generate plain-language explanations.

4. System Workflow

Healthcare Dataset → Data Cleaning → Exploratory Analysis → Feature Engineering → ML Model → SHAP Explanation → Dashboard → User Insights

For prediction:

Patient Input → Preprocessing → Trained ML Model → Risk Prediction → SHAP Explanation → Dashboard Result

5. FREE/Open-Source Tools & Technologies

Tool | Purpose

Python | Main programming language

Pandas | Data cleaning/manipulation

NumPy | Numerical operations

Matplotlib | Visualization

Seaborn | Statistical visualization

Scikit-learn | Machine learning

SHAP | Explainable AI

Streamlit | Dashboard/frontend

SQLite | Local database

Git/GitHub | Version control

Jupyter/VS Code | Development

Streamlit is free/open-source under Apache 2.0, and scikit-learn is open-source under the BSD-3-Clause license.

6. FREE/Open-Source LLM Models

Primary recommendation: Qwen3-4B

Model | Size | Recommendation | Why

Qwen3-4B | 4B parameters | Primary | Good balance of capability and

laptop/local deployment; Apache-2.0

Qwen3-1.7B | 1.7B | Backup | Lighter for laptops with limited RAM

Qwen3-8B | 8B | Optional | Better responses but requires more resources

Use of LLM: Only for converting dashboard/model results into simple explanations such as:

«“The patient's predicted risk is high. The strongest contributing factors were BMI and blood pressure.”»

Qwen3-4B is listed under the Apache-2.0 license and supports local use through common open-source tooling.

Local runtime: Ollama — MIT-licensed open-source software for running models locally.

Important: The LLM should not diagnose diseases or generate treatment recommendations. It should explain already-computed analytics.

7. FREE/Open-Source AI/ML Models/Algorithms

Primary ML model: Random Forest Classifier

Recommended options:

- Random Forest — primary; strong baseline and provides feature importance.

- Logistic Regression — simple, interpretable baseline.
- Decision Tree — highly interpretable and easy to demonstrate.
- XGBoost/LightGBM — optional advanced models if time permits.
- Neural Network/MLP — optional deep-learning extension; unnecessary for the MVP.

XAI:

- SHAP — explain individual predictions and overall feature importance.
- Feature importance — simple secondary explanation.

For a TY project, Random Forest + SHAP is the recommended combination because it demonstrates both ML and Explainable AI without excessive complexity.

Scikit-learn provides open-source implementations of classification algorithms including logistic regression, random forest, gradient boosting, and others.

8. Recommended Tech Stack

Frontend: Streamlit

Backend: Python

Database: SQLite

Data Processing: Pandas + NumPy

Visualization: Plotly + Matplotlib/Seaborn

AI/ML: Scikit-learn

XAI: SHAP

LLM: Qwen3-4B

LLM Runtime: Ollama

APIs: Local Python functions/REST endpoints only if required

Deployment: Local laptop; optional free Streamlit Community Cloud for the non-LLM dashboard

Architecture:

"Streamlit UI → Python Backend → SQLite/CSV → ML Model → SHAP → Results"

"Results → Optional Qwen3-4B → Natural-language explanation"

This stack avoids paid APIs and keeps the project realistic for a college laptop.

9. Dataset Requirements & Sources

Required data fields may include:

- Patient ID
- Age
- Gender
- BMI
- Blood pressure
- Glucose level

- Cholesterol
- Smoking status
- Physical activity
- Medical history
- Target health-risk category/outcome

Recommended sources:

- Kaggle healthcare datasets
- UCI Machine Learning Repository
- Government/public health datasets
- Other openly licensed datasets suitable for academic use

Prefer anonymized datasets with no names, phone numbers, addresses, medical record numbers, or other personally identifiable information.

Target size: approximately 1,000–10,000 records for the MVP.

10. Evaluation Metrics

ML Classification:

- Accuracy
- Precision
- Recall
- F1-score

- Confusion Matrix
- ROC-AUC, if applicable

Dashboard:

- Correctness of calculations
- Filter functionality
- Visualization readability
- Dashboard response time

XAI:

- Explanation consistency
- Feature importance relevance
- Correct mapping between model output and explanation

LLM:

- Factual consistency with model results
- No unsupported medical claims
- Human readability

11. MVP Scope

The minimum working product should contain:

- [] Healthcare CSV dataset
- [] Data cleaning/preprocessing
- [] Exploratory analysis
- [] Interactive Streamlit dashboard
- [] At least 5–7 useful visualizations
- [] Risk-category prediction
- [] Random Forest model
- [] SHAP explanation
- [] Basic patient-risk input form
- [] Model evaluation page
- [] GitHub project repository

LLM integration should be considered an optional MVP+ feature, not a dependency for the core dashboard.

12. Future Enhancements

- Real-time healthcare data integration
- More advanced ML/deep-learning models
- Personalized risk profiles
- Local LLM-powered natural-language reports
- Voice-based dashboard interaction
- Role-based access
- Mobile-friendly interface

- Time-series health monitoring
- Additional XAI techniques
- Integration with hospital systems, subject to privacy and security requirements

13. Feasibility & Estimated Resources

Technical feasibility: High

Recommended team: 2–4 students

Development duration: Approximately 6–10 weeks

Hardware:

- 8 GB RAM minimum for the core dashboard
- 16 GB RAM recommended if running Qwen3-4B locally
- 10–20 GB free storage
- Modern laptop/PC

Software cost: ₹0 using the recommended open-source stack.

Difficulty: Moderate and appropriate for TY B.Sc. Data Science.

Final Recommended Architecture

CSV/Open Dataset → Pandas → Random Forest → SHAP → Streamlit
Dashboard → Optional Qwen3-4B/Ollama Explanation